

Sensor-based Fuzzy Control of a 4-Legged Robot

Electrical Engineering Project on Intelligent Robot Control (M.Eng. Project)

Presented by: Adrian Iannantuono (300071774), 2024-03-22

Supervised by: Dr. Emil Petriu

School of Electrical Engineering and Computer Science

uOttawa.ca



uOttawa

Table of Contents

- Introduction
- Problem Definition
- Objectives
- Literature Survey
- Proposed Solution
 - Robot Design
 - Robot Movement
 - Simulation Overview
 - Ground Initialisation
 - Body Subsystem
 - Front & Rear Legs Subsystem
 - Stop Legs Controller
 - Body Controller
 - Fuzzy Logic System
- Results
- Conclusion
- References

Introduction

- How to mimic movement of a living organism to balance a robot travelling on uneven surfaces?
- Design a 4-legged robot with 6-degrees of freedom that can dynamically adjust its next movements based on its current foot position

FUZZY LOGIC

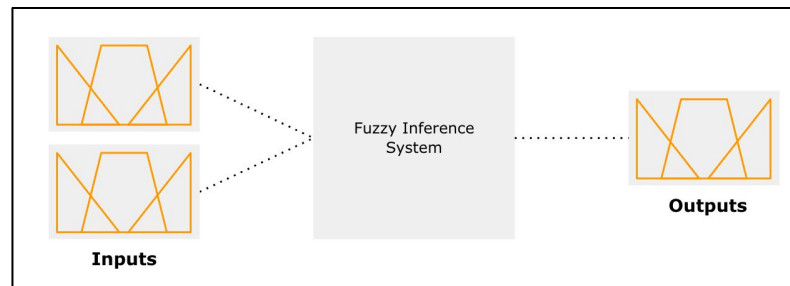


Fig 1. Fuzzy Logic

Problem Definition

- Create a 4-legged robot that is able to make smooth forward progress across an uneven surface while minimising the load on its motors
 - How does the robot balance?
 - How will the robot make forward progress?
 - How will the robot be controlled? (Fuzzy logic)

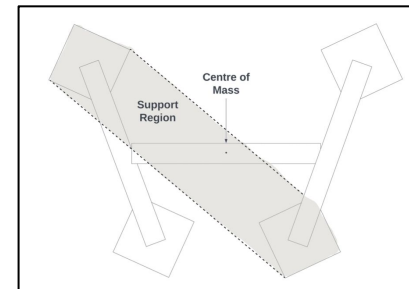


Fig 2. Robot centre of mass in support region

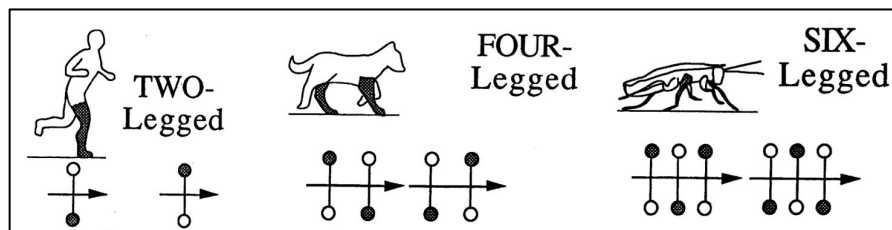


Fig 3. Animal gaits [1]

Objectives

- Research into 4 and 6-legged robot design
- Design and implement a 3D model of the 4-legged robot
- Kinematic model
 - Model the environment (interactions between the robot and the surface)
- Control the robots' movements (with and without the fuzzy logic controller)
- Run simulations and prepare results
- Compare simulation results of the robot with fixed programmed movements vs. the fuzzy logic controlled robot
 - Stability
 - Motor fatigue

Literature Survey

- Research was conducted on different techniques to achieve stable 6 and 4-legged robot movement
 - Simulations for a 6-legged fuzzy logic controlled robot were conducted based on [2] to aid in the design of a 4-legged robot
- [3] and [4] present 4-legged statically stable gaits on level surfaces through careful robot design and foot placement
- [5] presents a project creating a 4-legged robot using Simulink Multibody
- [6] presents a method of improving the accuracy of determining positions, velocities and accelerations of joints in robots
- [7] and [8] presents quadruped designs using inverse kinematics equations
- [9] implements a hexapod robot navigation algorithm using fuzzy logic
- [10] uses fuzzy logic to identify the terrain angle to control a prosthetic ankle
- [11] uses fuzzy logic to predict a robots instability when travelling on uneven surfaces

Proposed Solution

Robot Design

- 4 equally spaced legs with foot pads for stability
 - 2 leg sections (Front and Rear)
 - Mass located in robot body with negligible mass in legs
 - Foot pads ensure the centre of mass of the robot is in the region of support when walking
- Each leg has 2 degrees of freedom
 - Alpha (**Alpha1** and **Alpha2**)
- Beta (Beta1 and Beta2)
 - $\text{Beta1} = -\text{Beta2}$

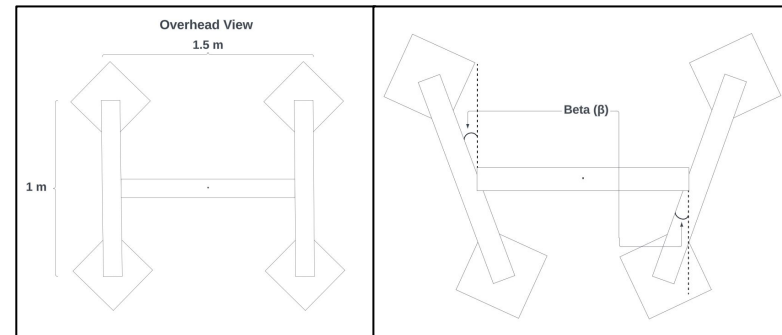


Fig 4. Robot overhead view

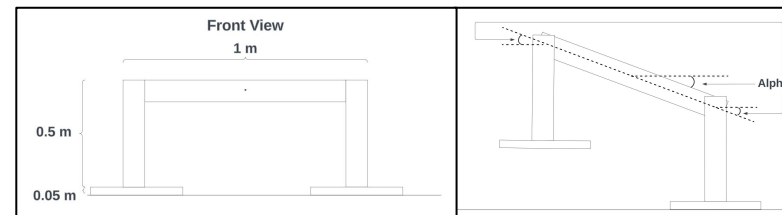


Fig 5. Robot front view

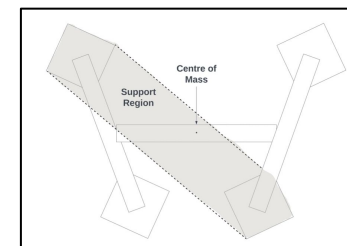
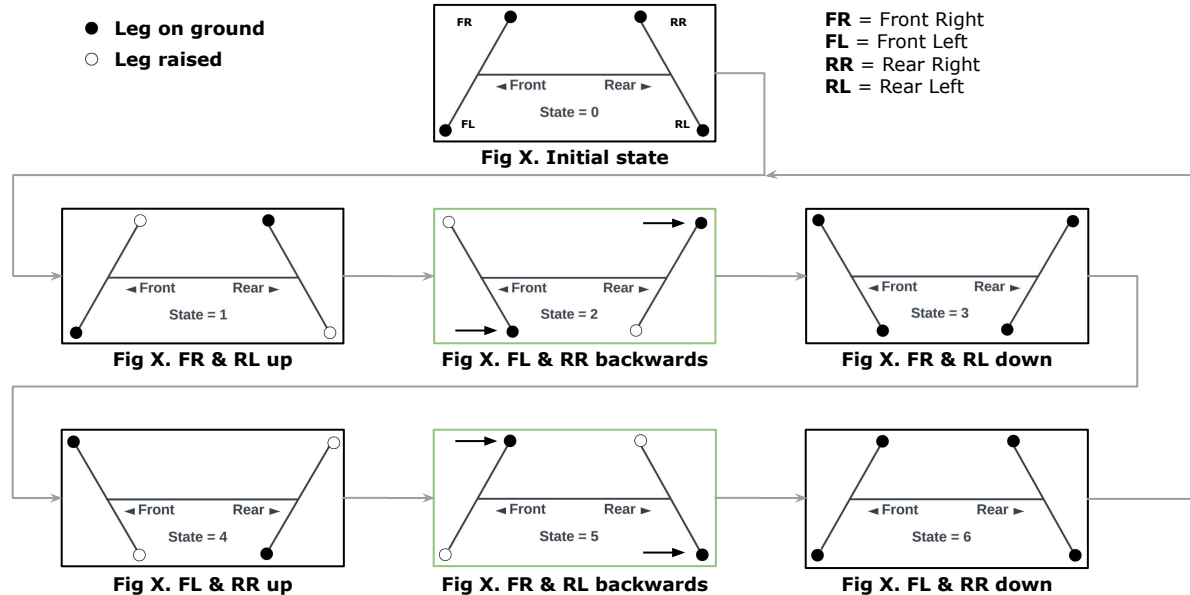


Fig 6. Robot centre of mass in support region

Proposed Solution

Robot Movement



Proposed Solution

Simulation Overview

- Created with MATLAB's Simulink Multibody tool
 - Design is highly modular
- Components:
 - 2 legs (**Front Legs & Rear Legs**)
 - 1 body
 - 1 grid surface
 - 2 controllers (**robotMovementController** & **stopLegsController**)
 - Measurements (position, angle, angular velocity, angular acceleration, motor torque)
- 100 second simulation
- Tools Used:
 - MATLAB (Simulink, Simscape Multibody, Fuzzy Logic Designer)

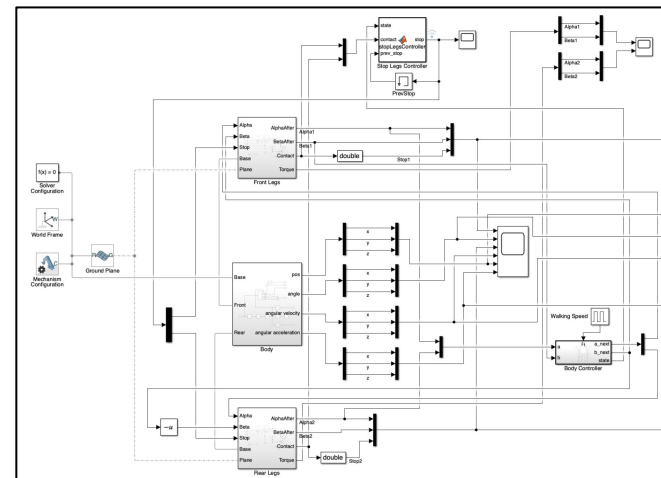


Fig 7. Overview

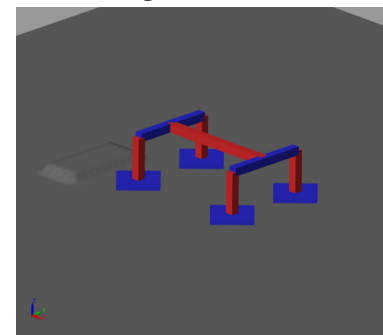


Fig 8. 4-Legged robot 3D simulation

Proposed Solution

Ground Initialisation

- To create a surface that is width x length (m), the **grid surface** component needs an NxM matrix containing the height z at every point on the surface
 - $N = (\text{width}/\text{resolution}) * 2 + 1$
 - $M = (\text{length}/\text{resolution}) * 2 + 1$
- To create a surface that is width x length (m), the **grid surface** component needs an NxM matrix containing the height z at every point on the surface
- Position of the object is specified through the **obstacle_pos** variable (left or right side of the robot)
- ground_z** stored as a global variable and read by the **grid surface** component

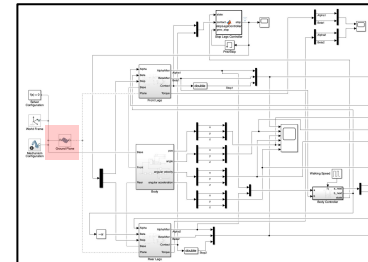


Fig 7. Overview

```

1 % Constants
2 OBSTACLE_HEIGHT = 0.10; % m
3 WIDTH = 5; % m
4 LENGTH = 5; % m
5 RESOLUTION = 0.1; % m
6
7 % Define the plane
8 ground_x = -WIDTH:RESOLUTION:WIDTH; % in m
9 ground_y = -WIDTH:RESOLUTION:WIDTH; % in m
10 ground_z = zeros(size(ground_x, 2), size(ground_y, 2)); % N x M (101 x 101)
11
12 % Set Obstacle
13 obstacle_size = OBSTACLE_HEIGHT;
14 obstacle_pos = "L"; % Left or right of robot
15
16 if obstacle_pos == "L"
17     ground_z(find(string(ground_x) == "-1.7"):find(string(ground_x) == "-1.3"), ...
18             find(string(ground_y) == "-1"):find(string(ground_y) == "0")) = obstacle_size;
19 elseif obstacle_pos == "R"
20     ground_z(find(string(ground_x) == "-1.5"):find(string(ground_x) == "-1.1"), ...
21             find(string(ground_y) == "0"):find(string(ground_y) == "1")) = obstacle_size;
22 end

```

Fig 9. init_ground.m

Proposed Solution

Body Subsystem

- Represents the main body of the robot (contains all the mass of the robot)
- Inputs:
 - **Base**
- Outputs:
 - **Front** (Transformed reference frame to front of body)
 - **Rear** (Transformed reference frame to rear of body)
 - Sensors
 - **pos**
 - **angle**
 - **angular velocity**
 - **angular acceleration**

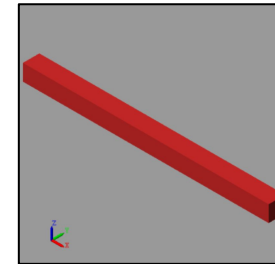


Fig 10. Body Geometry

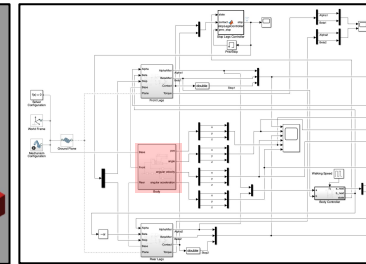


Fig 7. Overview

Body Dimensions (m)			Density (kg/m ³)	Weight (kg)
x	y	z		
1.5	0.1	0.1	5000	33.33

Table 1. Body Properties

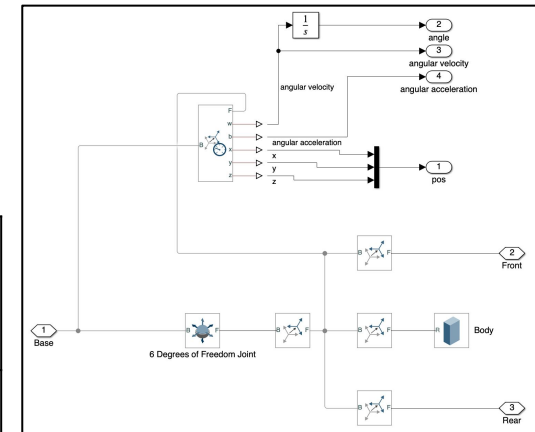


Fig 11. Body Subsystem

Proposed Solution

Front & Rear Legs Subsystem

- Represents each of the leg sections (negligible mass)
 - Includes lateral support arm, 2 vertical legs and 2 foot pads (includes contact sensors)
- Inputs:
 - Alpha (α)**
 - Beta (β)**
 - Stop**
 - Base**
 - Plane**
- Outputs:
 - AlphaAfter (α')**
 - BetaAfter (β')**
 - Contact**
 - Torque**

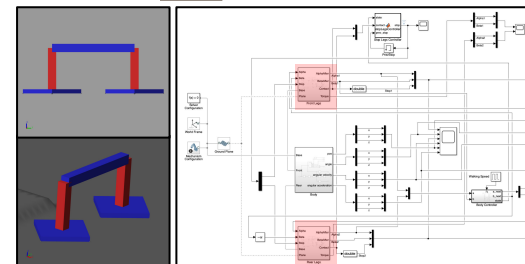


Fig 12. Leg Geometry

Fig 7. Overview

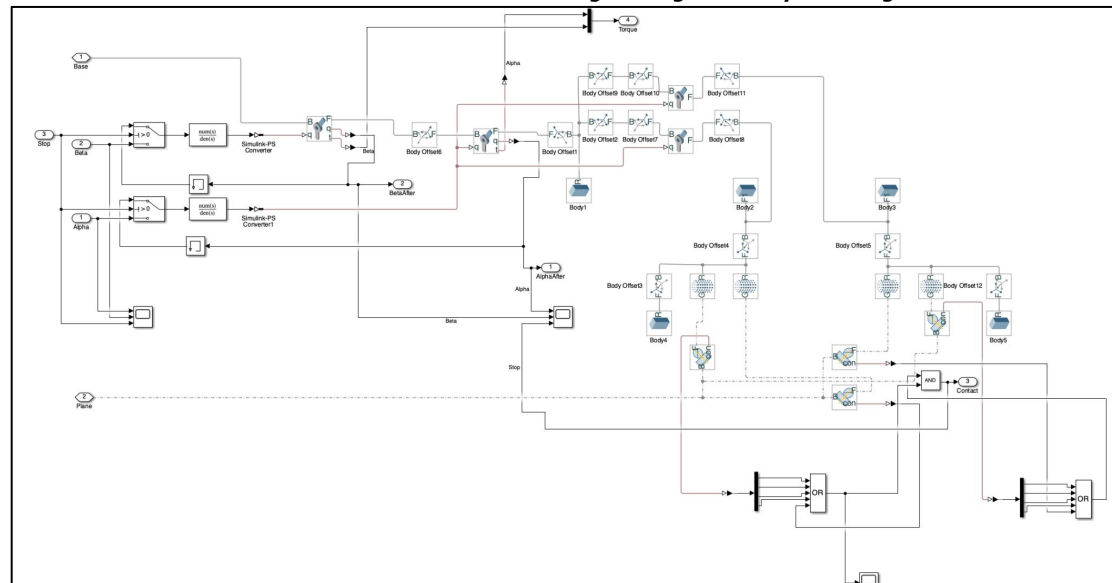


Fig 13. Front/Rear Legs Subsystem

Proposed Solution

Front & Rear Legs Subsystem (cont'd)

- Limiting abrupt changes of α and β is needed to maintain stability
- First order transfer function receives α or β which are step functions and limits abrupt changes in the input as seen in Fig X.
- K and τ were chosen to ensure the output reaches its final value within 2 seconds (period of robot movement)

$$H(s) = \frac{K}{\tau s + 1}, K = 1, \tau = 0.2$$

Equation 1. First order transfer function

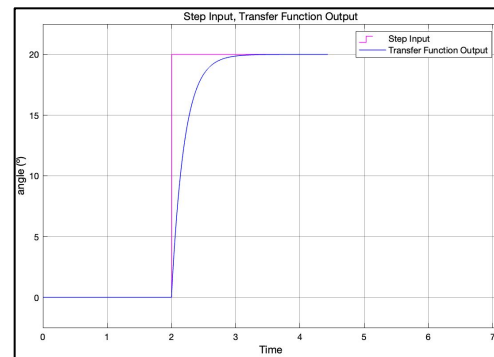


Fig 14. Step Input & Smoothing Transfer Function

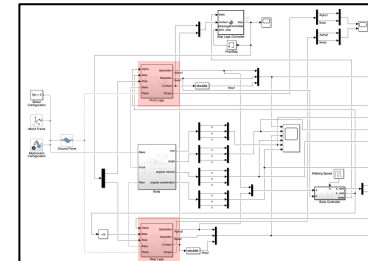


Fig 7. Overview

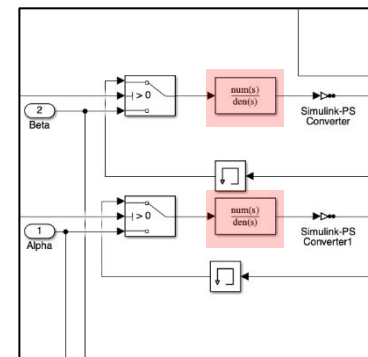


Fig 15. Front/Rear Legs Subsystem Zoomed

Proposed Solution

Front & Rear Legs Subsystem (cont'd)

- 4 Revolute joints
 - Rotates around their z-axis
- Joints actuated from input angle (force is automatically computed by MATLAB)
- **Alpha (α)** and **Beta (β)** joints have sensing outputs to determine the actual angle of each joint (angles are converted from radians to degrees)
 - **AlphaAfter**
 - **BetaAfter**
- Rotation of vertical legs relative to lateral leg support is needed to maintain level foot pads and better stability
 - Both of these joints should maintain an angle of **Alpha (α)**

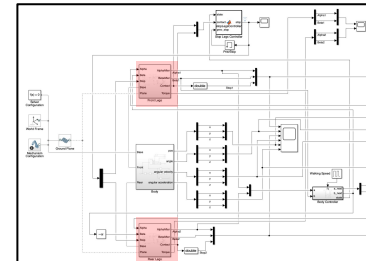


Fig 7. Overview

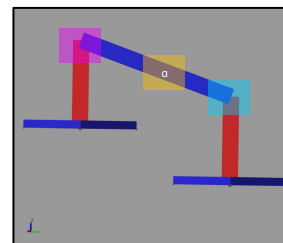


Fig 16. Side view of legs

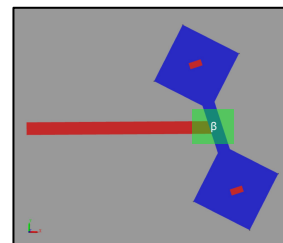


Fig 18. Top view of legs

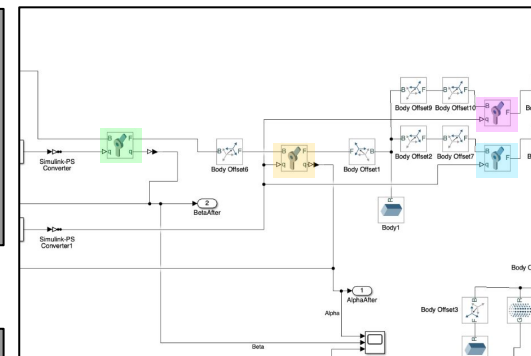


Fig 17. Front/Rear Legs Subsystem Zoomed

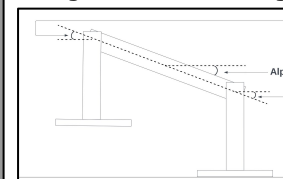


Fig 5. Side view Alpha

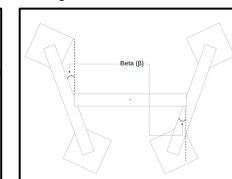


Fig 4. Side view Beta

Proposed Solution

Front & Rear Legs Subsystem (cont'd)

- Contact sensors on each leg
- Improves stability and allows for the controller to take action when both foot pads in one leg section is touching the ground
- Contact signal set to 1 if both foot pads are touching the ground

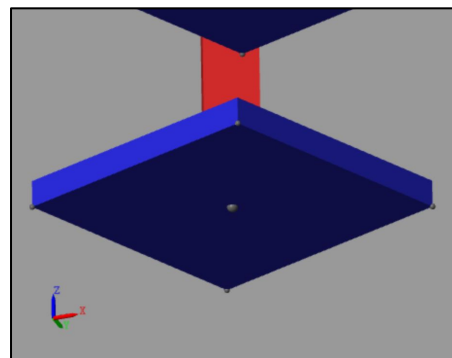


Fig 19. Contact sensors

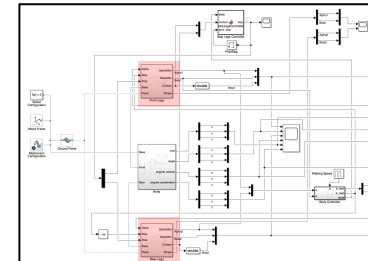


Fig 7. Overview

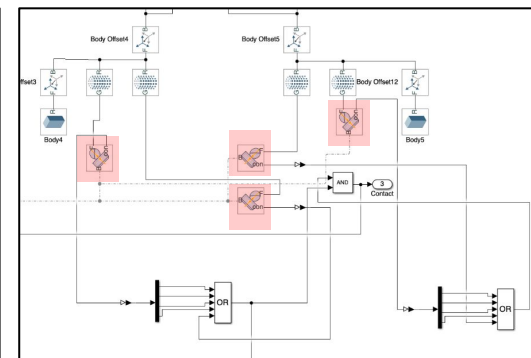


Fig 20. Front/Rear Legs Subsystem Zoomed

Proposed Solution

Stop Legs Controller

- Implements Finite State Machine (FSM) where state is externally determined
- Stops leg movement if both sides of front or rear legs makes contact with the ground
 - Used to maintain balance when walking over objects
- Inputs:
 - state**
 - contact = [front_contact; rear_contact];**
 - prev_stop = [front_stop; rear_stop];**
- Outputs:
 - stop = [front_stop; rear_stop];**

```

1 function stop = stopLegsController(state, contact, prev_stop)
2
3 % Legs are not stopped by default
4 stop = [0; 0];
5 % Element 1 = front legs
6 % Element 2 = rear legs
7
8 % Downstroke is on the 3rd and 6th state from the body controller FSM
9
10 switch state
11     case {3, 6}
12         % Down stroke front legs
13         if prev_stop(1) == 1
14             % If leg is not already stopped, check if it should be stopped
15             if contact(1)
16                 prev_stop(1) = 1;
17                 stop(1) = prev_stop(1);
18             else
19                 prev_stop(1) = 0;
20                 stop(1) = prev_stop(1);
21             end
22         else
23             % If leg is already stopped, keep it stopped while in this state
24             prev_stop(1) = 1;
25             stop(1) = prev_stop(1);
26         end
27
28         % Down stroke rear legs
29         if prev_stop(2) == 1
30             % If leg is not already stopped, check if it should be stopped
31             if contact(2)
32                 prev_stop(2) = 1;
33                 stop(2) = prev_stop(2);
34             else
35                 prev_stop(2) = 0;
36                 stop(2) = prev_stop(2);
37             end
38         else
39             % If leg is already stopped, keep it stopped while in this state
40             prev_stop(2) = 1;
41             stop(2) = prev_stop(2);
42         end
43     otherwise
44         % Legs are not stopped by default
45         stop = [0; 0];
46     end

```

Fig 21. Stop Legs Controller

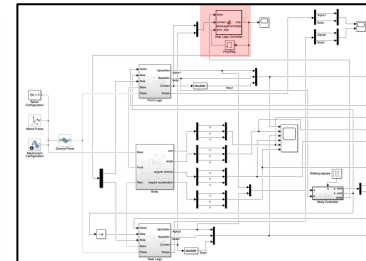


Fig 7. Overview

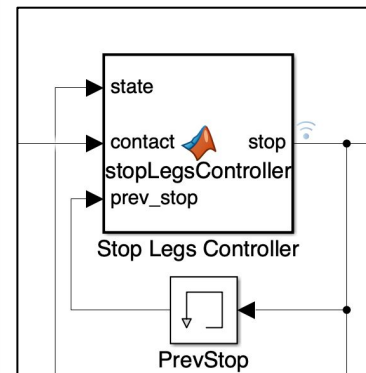


Fig 22. Stop Legs Controller Block

Proposed Solution

Body Controller

- Implements the main Finite State Machine (FSM) to actuate all joints in the robot
- Actuation can be done through **fixed programmed movements** or **fuzzy logic**
- 6 Stage movement design
- Speed controlled by a pulse generator (clock) with $T=4s$ and 50% pulse width
- Inputs:
 - Alpha (α) & Beta (β)**
 - stopped_a1 & stopped_a2**
 - state**
- Outputs:
 - a_next (α') & b_next (β')**
 - stopped_a1_next & stopped_a2_next**
 - state_next**

```

function [a_next, b_next, stopped_a1_next, stopped_a2_next, state_next] =
bodyControl(a, b, stopped_a1, stopped_a2, state)
1
2
3
4 ENABLE_FIS = 1;
5 fisFilename = 'quadRobotFIS_7';
6
7 a_next = [a(1); a(2)]; % Alpha angles
8 b_next = b; % Beta angles
9
10 % Set next stopped position to current stopped position
11 stopped_a1_next = stopped_a1;
12 stopped_a2_next = stopped_a2;
13
14 switch state
15 case 0
16     % Initial
17     a_next = [0; 0];
18     b_next = -20;
19     state_next = 1;
20 case 1
21     stopped_a1_next = a(1);
22     stopped_a2_next = a(2);
23     % FL/RL up
24     if ENABLE_FIS
25         fis = getFISCodeGenerationData(fisFilename);
26         output = evalfis(fis, [a(1) a(2) b]);
27         a_next = [output(1); output(2)];
28     else
29         a_next = [20; -20];
30     end
31     state_next = 2;
32 case 2
33     % FL/RL forward
34     b_next = 20;
35     state_next = 3;
36 case 3
37     % All down
38     a_next = [stopped_a1; stopped_a2]
39     b_next = [0; 0];
40     state_next = 4;
41 case 4
42     % FL/RR up
43     stopped_a1_next = a(1);
44     stopped_a2_next = a(2);
45     if ENABLE_FIS
46         fis = getFISCodeGenerationData(fisFilename);
47         output = evalfis(fis, [a(1) a(2) b]);
48         a_next = [output(1); output(2)];
49     else
50         a_next = [-20; 20];
51     end
52     state_next = 5;
53 case 5
54     % FL/RR forward
55     b_next = -20;
56     state_next = 6;
57 case 6
58     % All down
59     a_next = [stopped_a1; stopped_a2]
60     b_next = [0; 0];
61     state_next = 1;
62 otherwise
63     state_next = 0;
64 end
    
```

Fig 23. Body Controller

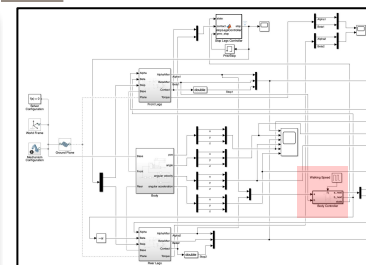


Fig 7. Overview

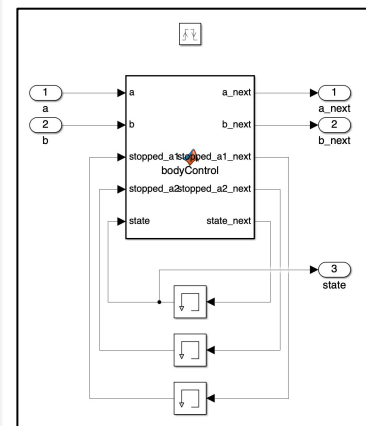


Fig 24. Body Controller Subsystem

Proposed Solution

Fuzzy Logic System

- Mamdani Type-1 system
- Adapted from [2] to fit this projects design
- Inputs:
 - **Alpha1 (α_1) - Range $[-20^\circ$ to $20^\circ]$**
 - **Alpha2 (α_2) - Range $[-20^\circ$ to $20^\circ]$**
 - **Beta (β) - Range $[-20^\circ$ to $20^\circ]$**
- Outputs:
 - **Alpha1' (α_1') - Range $[-20^\circ$ to $20^\circ]$**
 - **Alpha2' (α_2') - Range $[-20^\circ$ to $20^\circ]$**

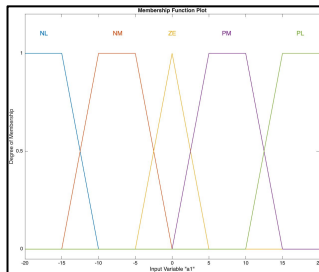


Fig 25. Input angle Alpha 1 or 2

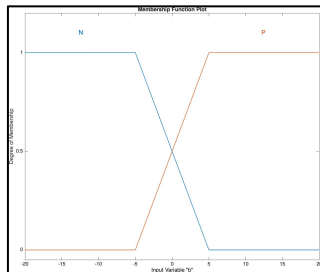


Fig 26. Input angle Beta

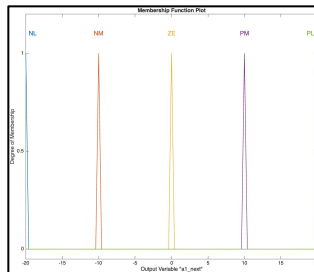


Fig 27. Output angle α_1' or α_2'

α_1'/α_2'		Beta (β)	
		N	P
α_1 or α_2	ZE	PL/NL	NL/PL

Table 2. Fuzzy rules with robot on level surface [2]

α_1'/α_2'		Beta (β)	
		N	P
Alpha1 (α_1)	NL	NM/NL	NL/PL
	NM	ZE/NM	NL/PL
	PM	PL/NL	ZE/PM
	PL	PL/NL	PM/PL

Table 3. Fuzzy rules with front leg on object [2]

α_1'/α_2'		Beta (β)	
		N	P
Alpha2 (α_2)	NL	PL/NL	NL/NM
	NM	PL/NL	NM/ZE
	PM	PM/ZE	NL/PL
	PL	PL/PM	NL/PL

Table 4. Fuzzy rules with rear leg on object [2]

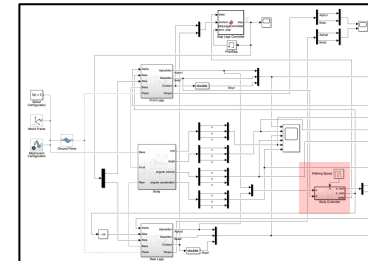


Fig 7. Overview

Proposed Solution

Fuzzy Logic System (cont'd)

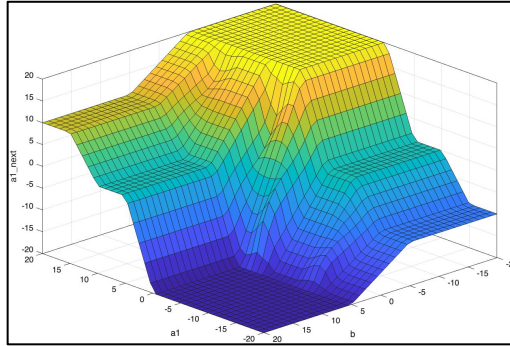


Fig 28. Control surface for Alpha1 vs. Beta vs. Alpha1'

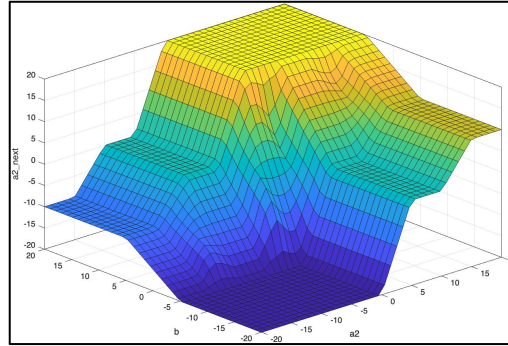


Fig 29. Control surface for Alpha2 vs. Beta vs. Alpha2'

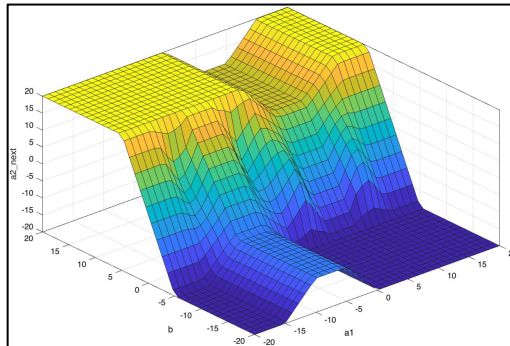


Fig 30. Control surface for Alpha1 vs. Beta vs. Alpha2'

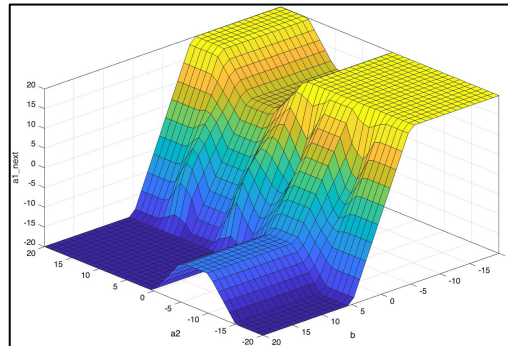


Fig 31. Control surface for Alpha2 vs. Beta vs. Alpha1'

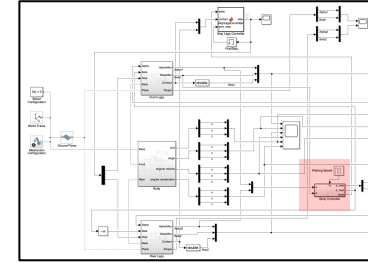


Fig 7. Overview

Results

Fixed Programmed Movement



Simulate

vs.

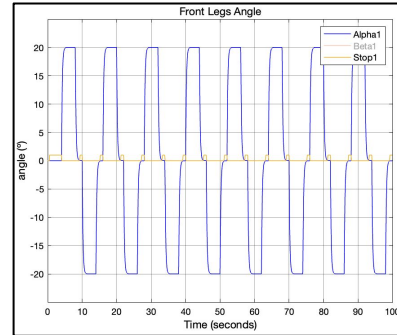
**Fuzzy Logic Controlled
Movement**



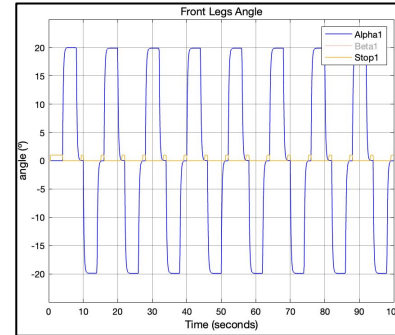
Simulate

Results

Level Surface

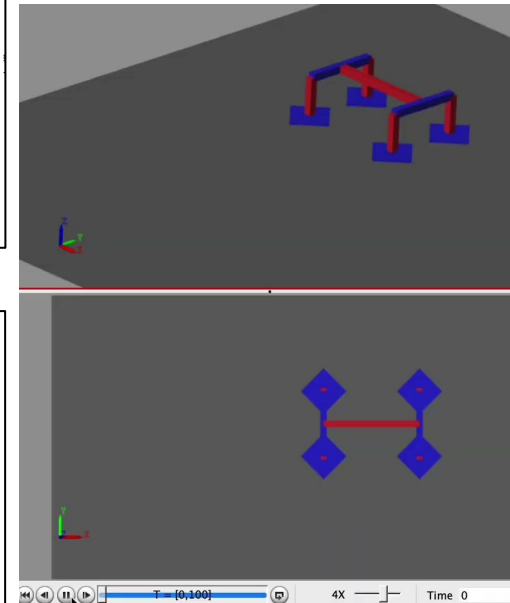
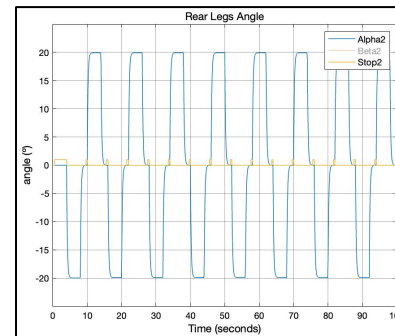
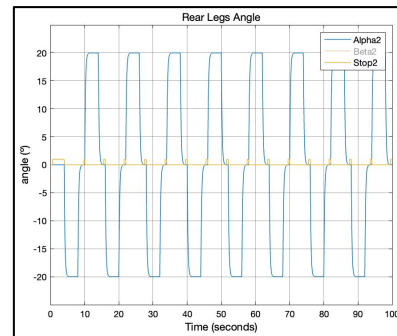


Fixed Programmed Movement



Fuzzy Logic Controlled Movement

vs.



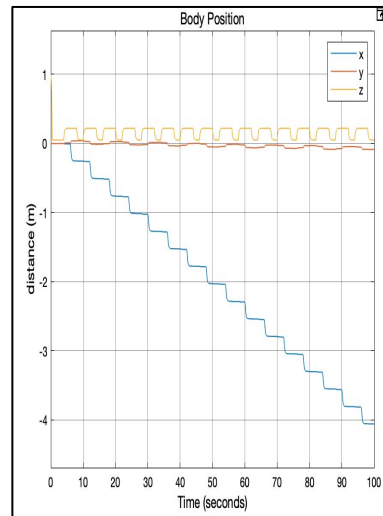
Video 2. Fuzzy logic controlled movements on level ground

Video 1. Fixed Programmed movements on level ground

Results

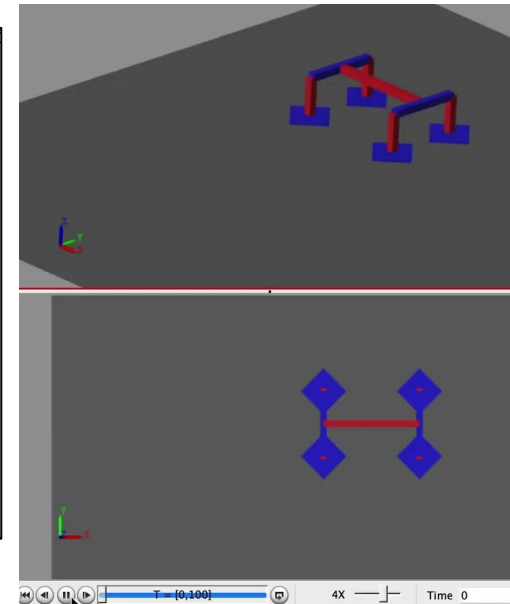
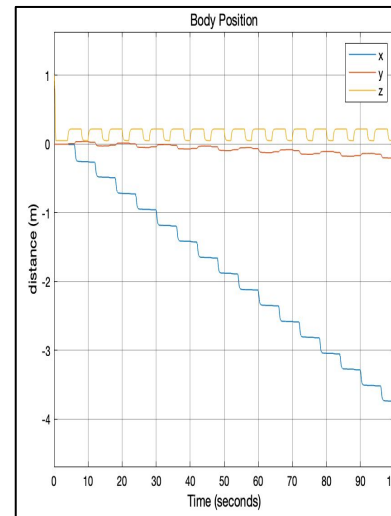
Level Surface (cont'd)

Fixed Programmed Movement



vs.

Fuzzy Logic Controlled Movement

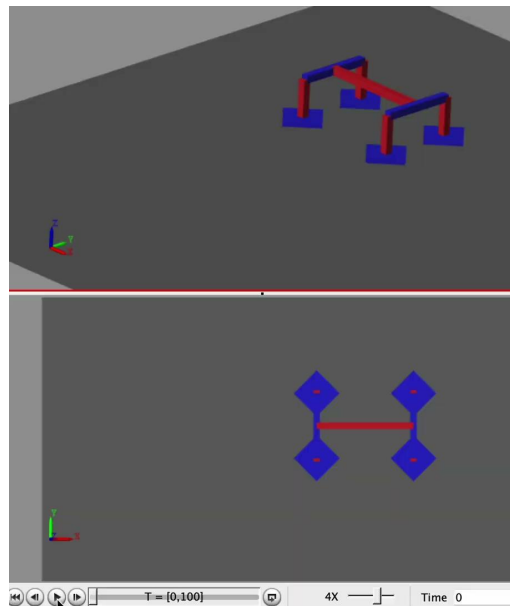


Video 1. Fixed Programmed movements on level ground

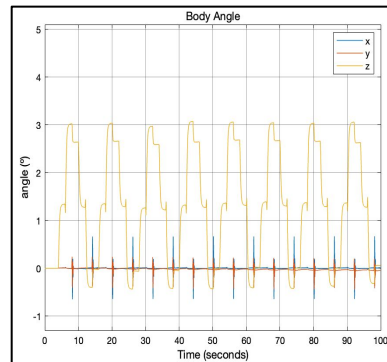
Video 2. Fuzzy logic controlled movements on level ground

Results

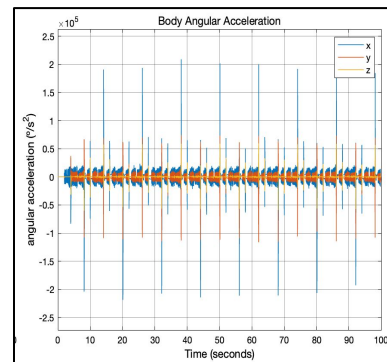
Level Surface (cont'd)



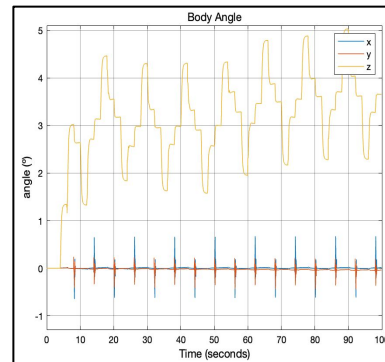
Video 1. Fixed Programmed movements on level ground



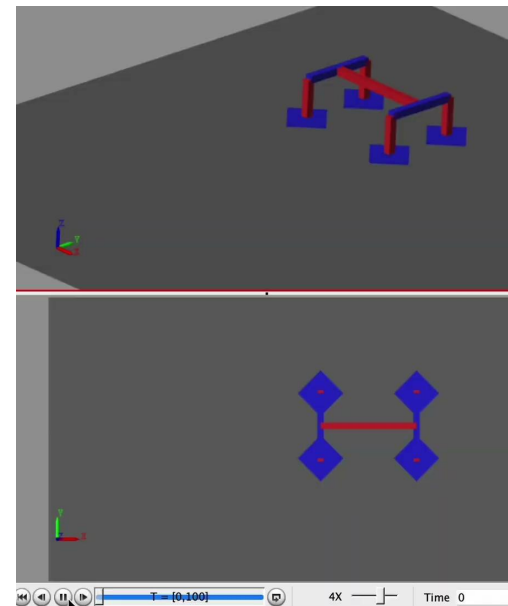
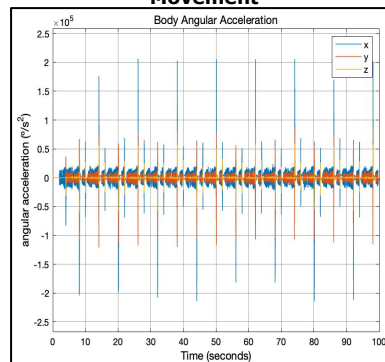
Fixed Programmed Movement



vs.



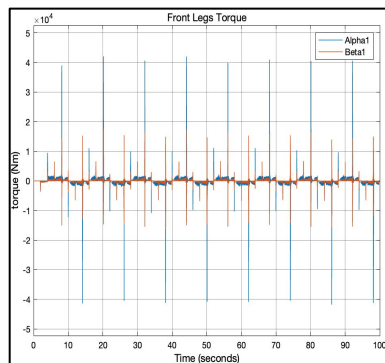
Fuzzy Logic Controlled Movement



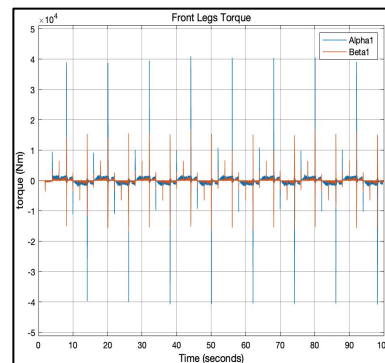
Video 2. Fuzzy logic controlled movements on level ground

Results

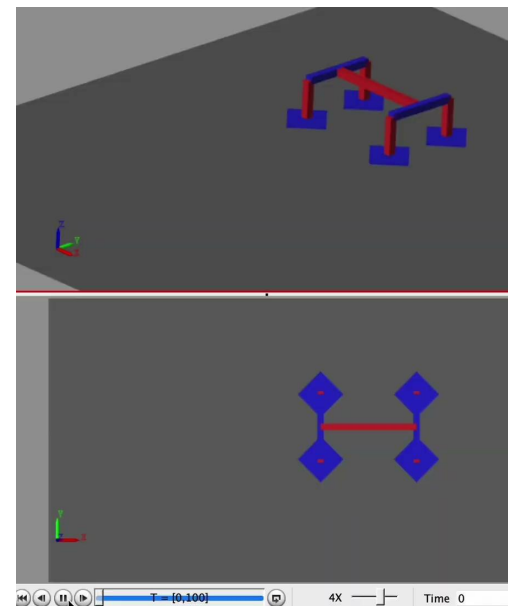
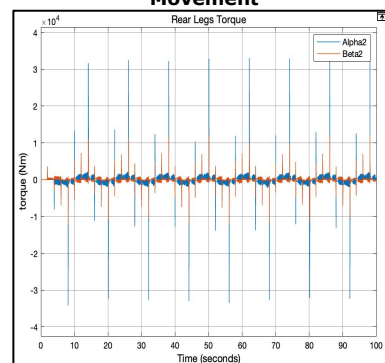
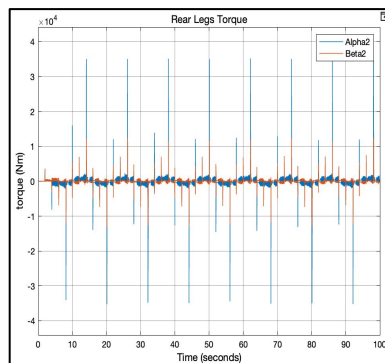
Level Surface (cont'd)



Fixed Programmed Movement



Fuzzy Logic Controlled Movement

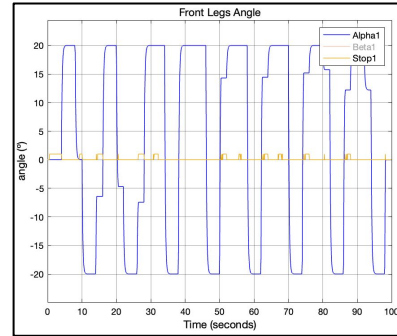


Video 2. Fuzzy logic controlled movements on level ground

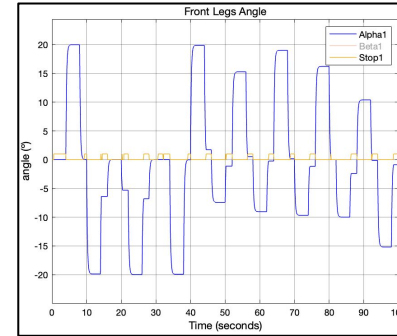
Video 1. Fixed Programmed movements on level ground

Results

Object Under Left Legs

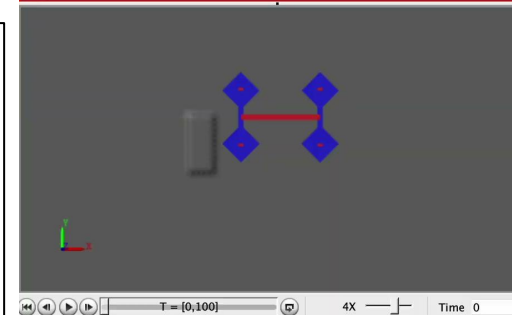
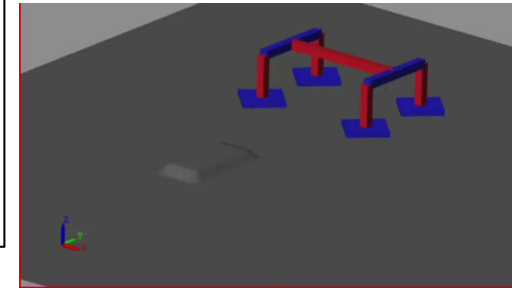
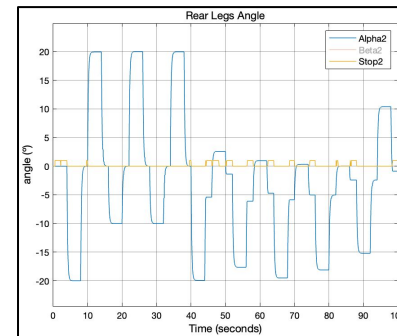
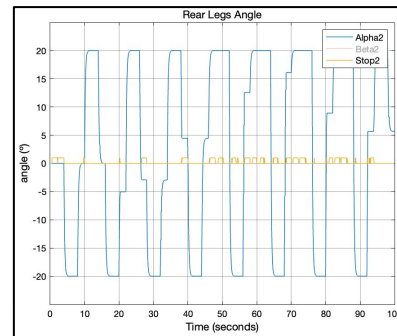


Fixed Programmed Movement



Fuzzy Logic Controlled Movement

vs.



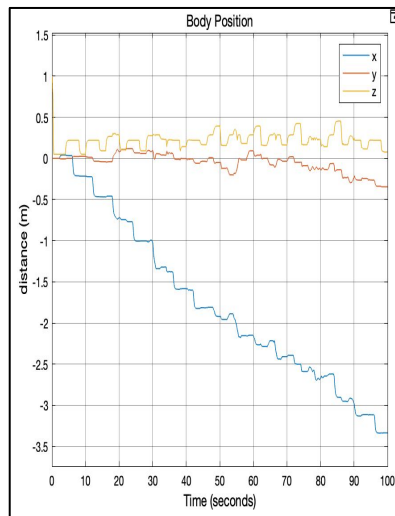
Video 3. Fixed Programmed movements with obstacle on left side

Video 4. Fuzzy logic controlled movements with obstacle on left side

Results

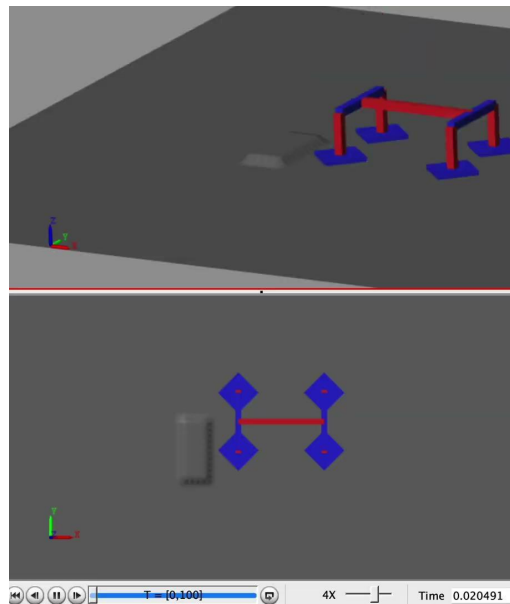
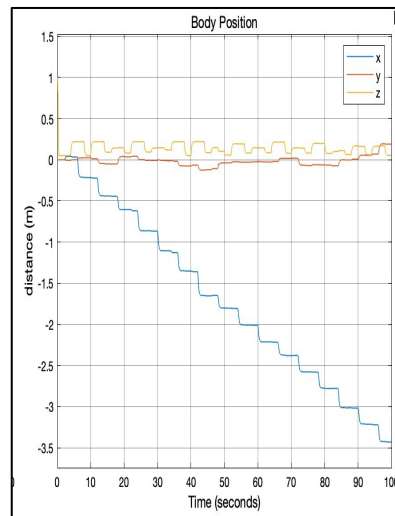
Object Under Left Legs (cont'd)

Fixed Programmed Movement

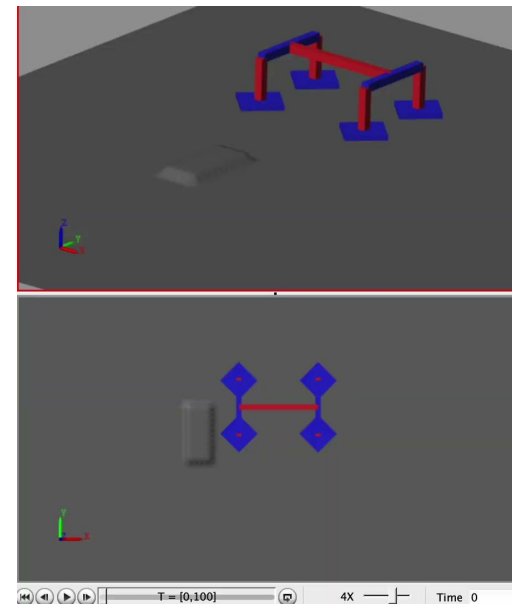


vs.

Fuzzy Logic Controlled Movement



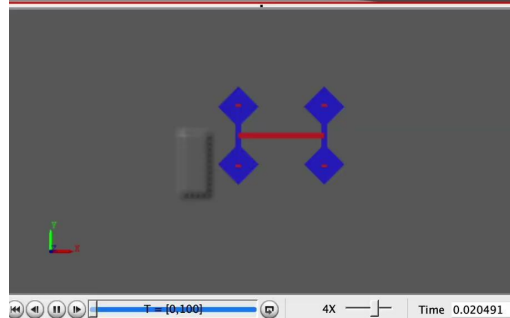
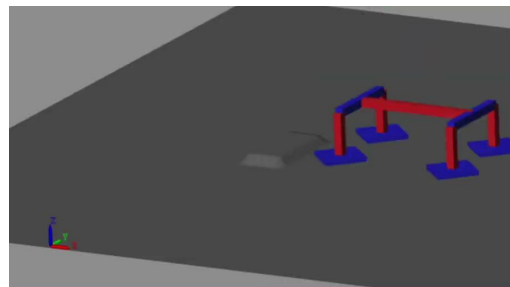
Video 3. Fixed Programmed movements with obstacle on left side



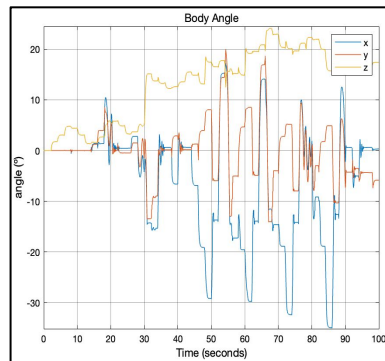
Video 4. Fuzzy logic controlled movements with obstacle on left side

Results

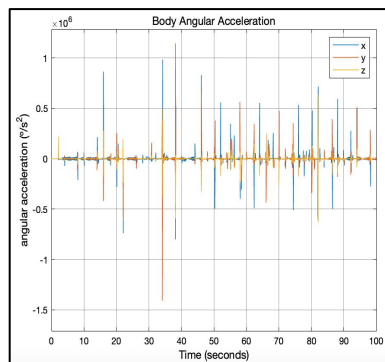
Object Under Left Legs (cont'd)



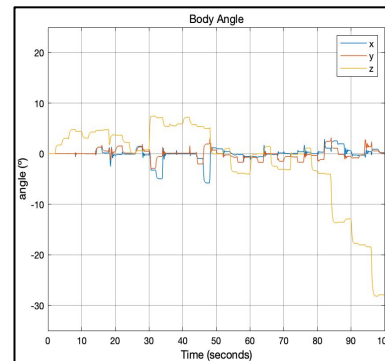
Video 3. Fixed Programmed movements with obstacle on left side



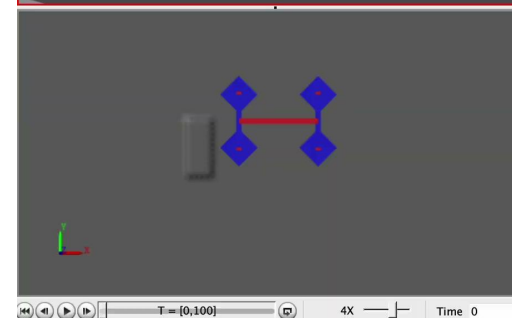
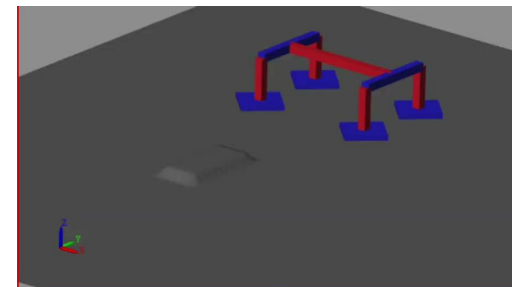
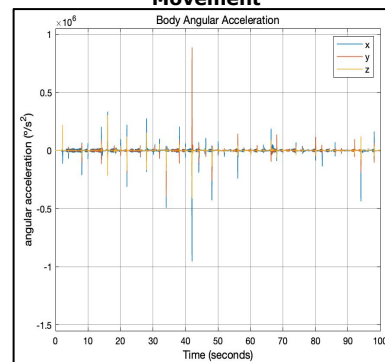
Fixed Programmed Movement



vs.



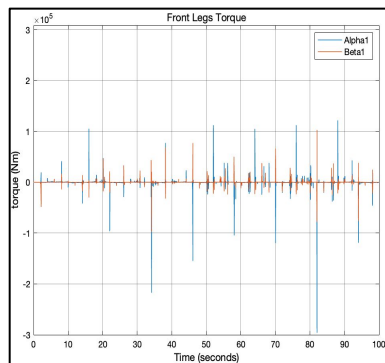
Fuzzy Logic Controlled Movement



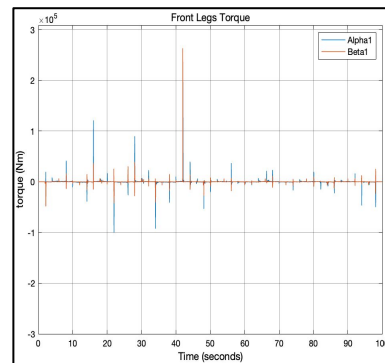
Video 4. Fuzzy logic controlled movements with obstacle on left side

Results

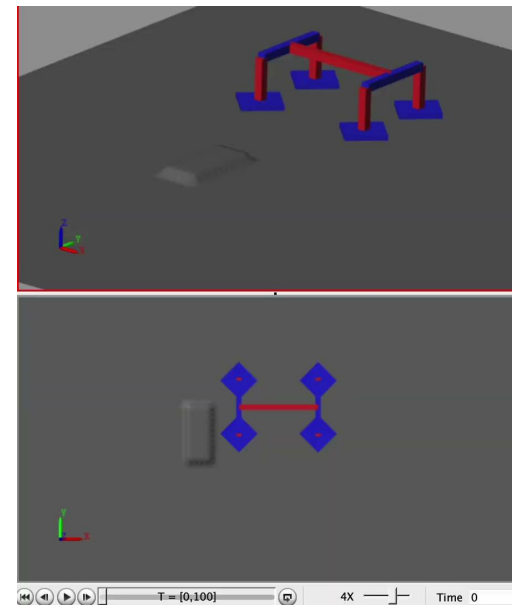
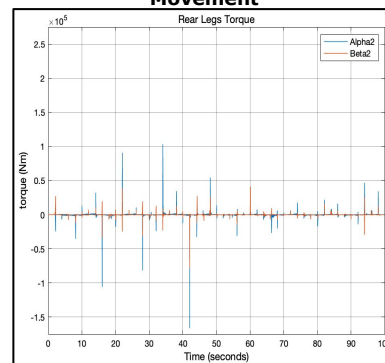
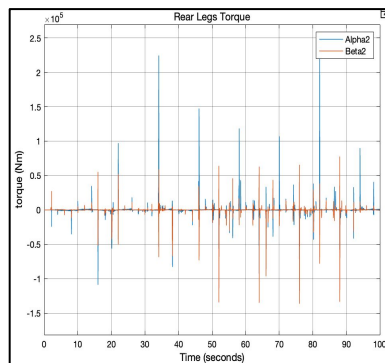
Object Under Left Legs (cont'd)



Fixed Programmed Movement



Fuzzy Logic Controlled Movement

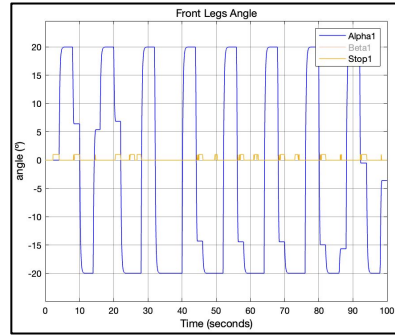


Video 4. Fuzzy logic controlled movements with obstacle on left side

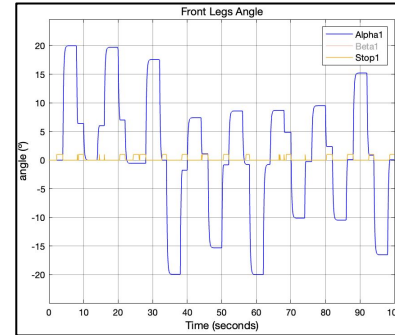
Video 3. Fixed Programmed movements with obstacle on left side

Results

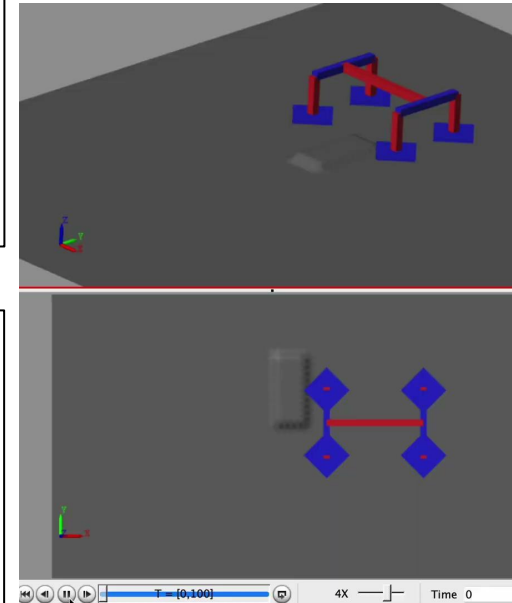
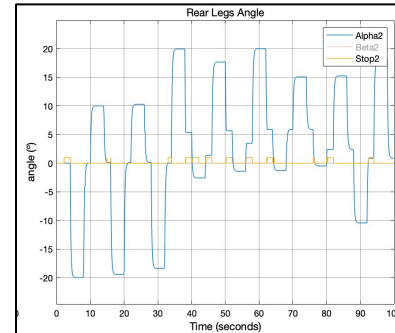
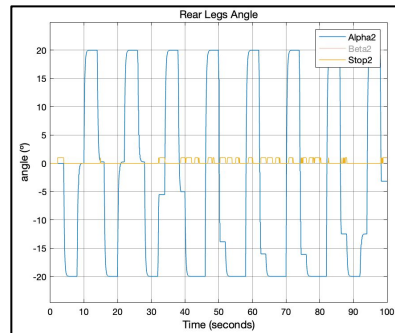
Object Under Right Legs



Fixed Programmed Movement



Fuzzy Logic Controlled Movement



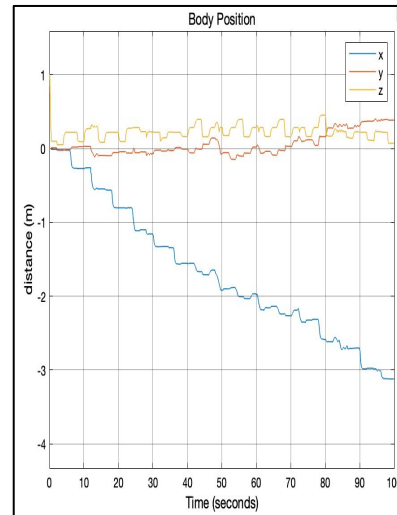
Video 6. Fuzzy logic controlled movements with obstacle on right side

Video 5. Fixed Programmed movements with obstacle on right side

Results

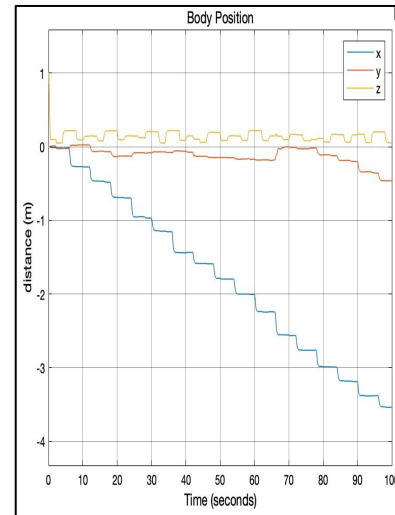
Object Under Right Legs (cont'd)

Fixed Programmed Movement

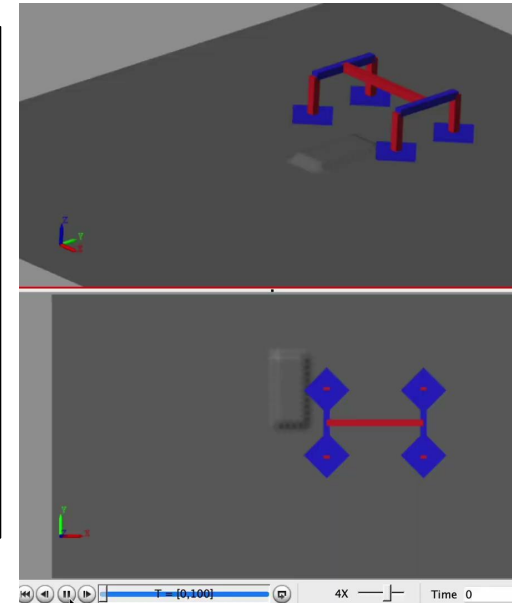


vs.

Fuzzy Logic Controlled Movement



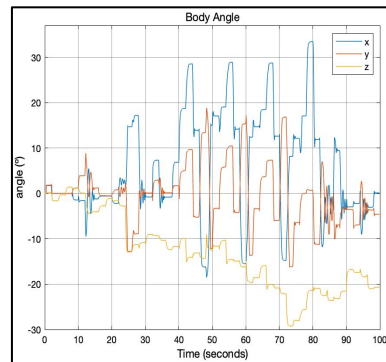
Video 5. Fixed Programmed movements with obstacle on right side



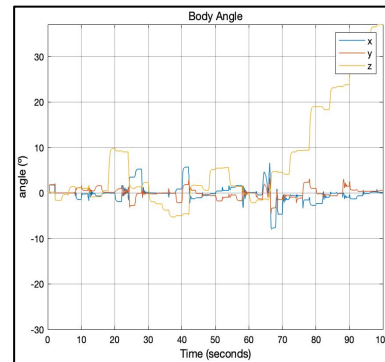
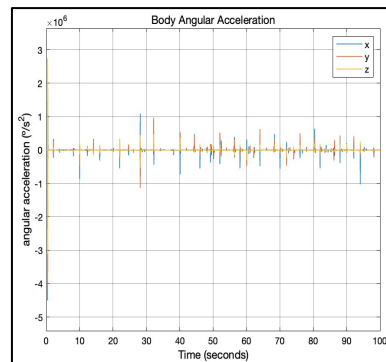
Video 6. Fuzzy logic controlled movements with obstacle on right side

Results

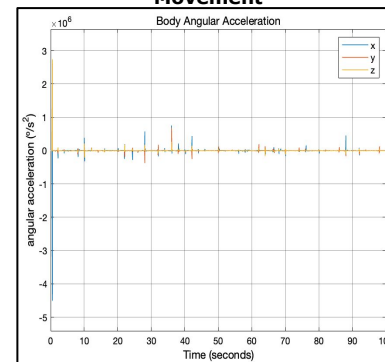
Object Under Right Legs (cont'd)



Fixed Programmed Movement

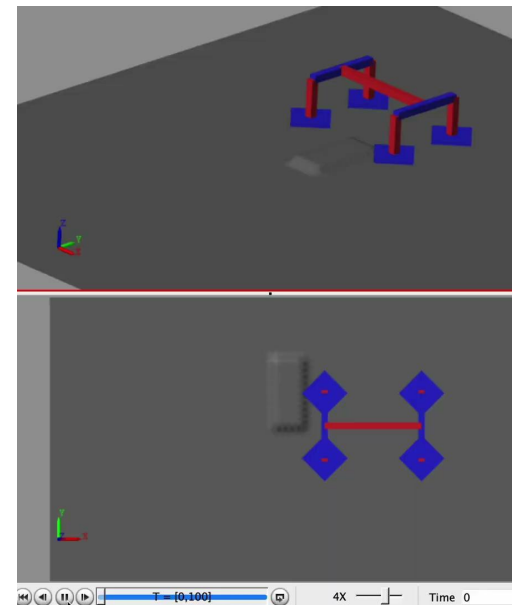


Fuzzy Logic Controlled Movement



vs.

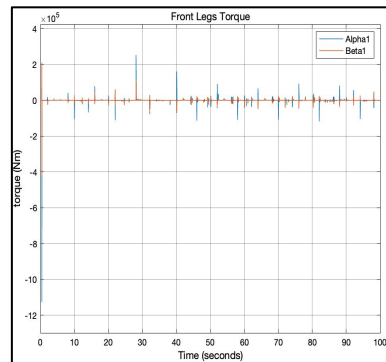
Video 5. Fixed Programmed movements with obstacle on right side



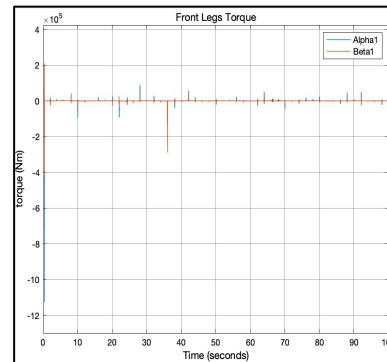
Video 6. Fuzzy logic controlled movements with obstacle on right side

Results

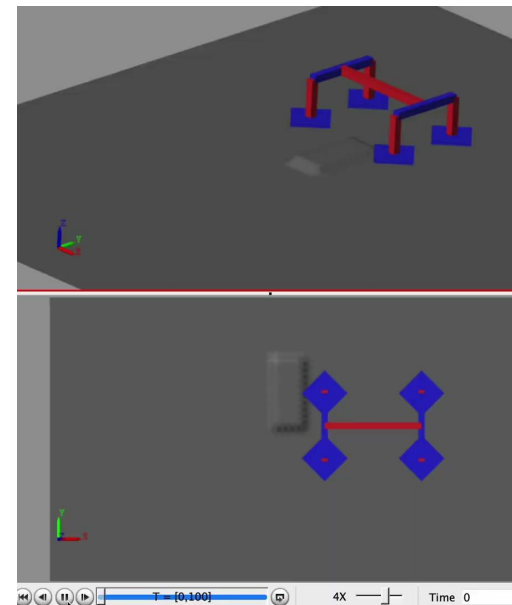
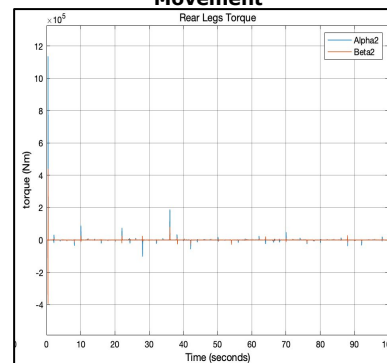
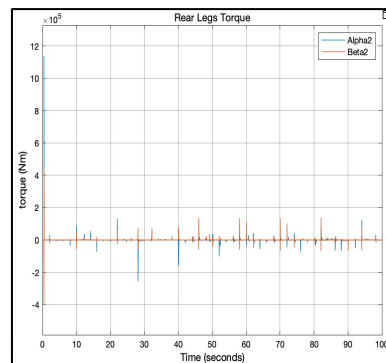
Object Under Right Legs (cont'd)



Fixed Programmed Movement



Fuzzy Logic Controlled Movement



Video 6. Fuzzy logic controlled movements with obstacle on right side

Video 5. Fixed Programmed movements with obstacle on right side

Conclusion

- The fuzzy logic controlled robot had improved stability when compared to a fixed programmed movement design
- Fuzzy logic is an effective tool to improve the stability of a robot traveling on an uneven surface
- Fuzzy logic controlled movements allowed for the robots motors to be under less stress and use less power to perform the same task

References

- [1] Full, R.J., Farley, C.T., Winters, J.M. (2000). Musculoskeletal Dynamics in Rhythmic Systems: A Comparative Approach to Legged Locomotion. In: Winters, J.M., Crago, P.E. (eds) *Biomechanics and Neural Control of Posture and Movement*. Springer, New York, NY.
https://doi.org/10.1007/978-1-4612-2104-3_13
- [2] Z. Sakr and E. M. Petriu, "Hexapod Robot Locomotion using a Fuzzy Controller," *2007 International Workshop on Robotic and Sensors Environments*, Ottawa, ON, Canada, 2007, pp. 1-5, doi: 10.1109/ROSE.2007.4373969.
- [3] A. Globisz and D. Krawczyk, "Statically Stable Gait Synthesis Algorithm For 4-legged Walking Robot Based On The Ground Reaction Forces Determination," *2018 19th International Conference on Research and Education in Mechatronics (REM)*, Delft, Netherlands, 2018, pp. 81-86, doi: 10.1109/REM.2018.8421774.
- [4] H. Ishihara and K. Kuroi, "A Four-Leg Locomotion Robot for Heavy Load Transportation," *2006 IEEE/RSJ International Conference on Intelligent Robots and Systems*, Beijing, China, 2006, pp. 5731-5736, doi: 10.1109/IROS.2006.282379.
- [5] A.K. Mishra, A. Chauhan, U. Vashishtha, "Simulation of Quadruped Robot," Shahbad Daultapur, Delhi

References

- [6] D. Neculescu and R. Jassemi-Zargani, "Extended Kalman filter based sensor fusion for operational space control of a robot arm," *IMTC 2001. Proceedings of the 18th IEEE Instrumentation and Measurement Technology Conference. Rediscovering Measurement in the Age of Informatics (Cat. No.01CH 37188)*, Budapest, Hungary, 2001, pp. 915-918 vol.2, doi: 10.1109/IMTC.2001.928212.
- [7] B. Muntean, M. Zaha and S. Grigorescu, "Analytic Inverse Kinematics Model and Trajectory Planning for an 18 DoF Quadruped Robot," *2023 27th International Conference on System Theory, Control and Computing (ICSTCC)*, Timisoara, Romania, 2023, pp. 277-282, doi: 10.1109/ICSTCC59206.2023.10308451.
- [8] P. B, T. K, R. K. M, Y. S, R. P. J and P. B, "Design and Development of a Quadruped Robot for achieving Multi-Terrain Locomotion," *2023 Second International Conference on Trends in Electrical, Electronics, and Computer Engineering (TEECCON)*, Bangalore, India, 2023, pp. 51-56, doi: 10.1109/TEECCON59234.2023.10335811.
- [9] M. Kern and P.-Y. Woo, "Implementation of a hexapod mobile robot with a fuzzy controller," *Robotica*, vol. 23, no. 6, pp. 681-688, 2005. doi:10.1017/S0263574705001682
- [10] M. Chang, K. Kim and D. Jeon, "Research on Terrain Identification of the Smart Prosthetic Ankle by Fuzzy Logic," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 27, no. 9, pp. 1801-1809, Sept. 2019, doi: 10.1109/TNSRE.2019.2933874.

References

[11] F. Zha, Q. Wang, P. Zheng, C. Chen and W. Guo, "Research on Dynamic Stability Prediction of Large Heavy Six-legged Robots On Slope," *2018 11th International Workshop on Human Friendly Robotics (HFR)*, Shenzhen, China, 2018, pp. 7-12, doi: 10.1109/HFR.2018.8633528.

Thank you